

# RWE's ETL pipeline with picard : : CHEAT SHEET



## Pipeline

- **pipeline\$new(config\_pipeline):** boots the orchestrator, loads the main YAML and caches the ordered list of scripts so every run starts from a reproducible configuration snapshot.
- **pipeline\$run():** iterates through the configured steps (or a user-provided subset), sources each script, and stops the pipeline with a descriptive error if any step fails.

### config\_pipeline.yaml

```
# -----
# Common elements
# -----
common:
  dir_commonfunctionality: ...
  dir_meta_data: ...
  sga_cat: ...
  quality_pregnancy: ...

# -----
# Step of the pipeline
# -----
steps:
  T2: run/run_T2_pipeline.R
  T3: run/run_T3_pipeline.R
  T4: run/run_T4_pipeline.R
  T5: run/run_T5_pipeline.R

# -----
# Databases
# -----
databases:
  dir_d2_db: data/D2_cdm/d2.duckdb
  dir_concept: ...
  dir_preprocess_concept: ...
  dir_all_concepts: ...
  dir_anchored_variables_db: ...duckdb
```

### config\_T2.yaml

```
# -----
# T2 directories
# -----
T2:
  name: "semantic_harmonization"
  root:
  intermediate:

# -----
# T2 Semantic Harmonization Configuration
# -----
cdm_table_names: ["PERSONS", "VACCINES",
  "OBSERVATION_PERIODS",
  "MEDICAL_OBSERVATIONS", "MEDICINES",
  "EVENTS", "PROCEDURES",
  "SURVEY_OBSERVATIONS"]

# -----
# T2 Substeps
# -----
substep:
  start_up_t2: true
  set_db: true
  create_dapspecificcodelist: true
  create_concepts_database: true
  processing_persons: true
  create_spells: true
  processing_pregnancy: true
  process_vxrsv: true
  preprocessing_concepts_database: true
  merge_concepts_database: true
  get_counts_all_concepts: true
```

## Input & Output

- **load(file\_path, file\_name, col\_types, ...)**  
normalizes the target path, resolves case-mismatch issues, picks the right reader based on extension, and always returns a data.table, so every script can rely on consistent casing and column types.
- **save(data, file\_path, ..., create\_plot)**  
mirrors read\_data() in reverse: it enforces directories, dispatches to the correct writer, and can auto-generate feature plots (with privacy-sensitive IDs dropped) alongside the saved dataset.
- **load\_sql\_query(file\_path, params, safe\_interpolation)**  
ingests a .sql file, cleans tabs and comments, and interpolates {placeholders} only when every parameter is supplied, making templated SQL safe to reuse across DAPs.
- **execute\_sql\_file(sql, conn, execute)**  
runs or simply returns an interpolated SQL string; it auto-detects whether to use dbGetQuery() or dbExecute(), so ETL scripts can switch between SELECT and DDL/DML without extra branching.
- **plot\_data\_features(data, file\_path, ...)**  
bucket columns by type, build histograms/bar charts with summary stats, and persist PNGs per dataset.
- **load\_config\_values(file\_path)**  
Either reads a single YAML or bulk-loads every configuration/\*.yaml into the global environment with lowercase names, making it easy to pull project metadata without re-parsing files.

# RWE's ETL pipeline with picard :: CHEAT SHEET



## Auditing

- **audit\_start(dir\_output, file\_name, deap\_name, delete\_old)**  
Creates or refreshes a timestamped audit text file, writes a header with the DEAP identifier, and caches the filepath in options so subsequent calls know where to append.
- **audit\_add(...)**  
Appends arbitrary status text (row counts, filter criteria, etc.) to the active audit file, letting analysts narrate every transformation in plain language alongside the automated logs.
- **audit\_end()**  
Stamps the audit with finish time, elapsed duration, and the package release version (via Git tags) before clearing the cached state—closing the loop for traceable handovers.

## Logging

- **logger\_manager\$configure(log\_dir)**  
initializes the global logger, rotates old log files, and ensures every logger::log\_\* call is duplicated to console and disk.
- **logger\_manager\$init\_step\_logger(step\_name)**  
spawns a step-specific log sink so verbose DEBUG
- **run\_script(file\_path, log\_dir, registry\_path)**  
Executes an arbitrary R script, records its SHA1 hash plus timestamp into a registry log, and exposes the environment/timing details if you need to inspect what was produced.

## Miscellaneous

- **set\_dates(df, date\_cols, date\_format, reference\_date)** coerces selected columns to Date for handling character, numeric, or integer inputs, so downstream computations never mix date encodings.

## Typical sub-step script

```
# Project/transformations/T2_semantic_harmonization/source_code/xxx.R

### Initial setup
logger::log_info("[Processing xxx] - START")

picard::load_config_values()

...

picard::audit_start(deap_name = "xxx")

dt <- picard::read_data(
  file_path = file_path,
  file_name = "file_name.rds"
)
...

picard::audit_add(
  paste(
    "now dt has: ", nrow(persons)
  )
)

picard::save_data(
  data = dt,
  file_path = file_path,
  file_name = "file_namefst",
  create_plot = TRUE
)

picard::audit_end()
```